

DS Project Scope

Kushagra Trivedi (2023101048) Ronit Jaliha (2023101028) Prasoon Dev (2023111014)

1 Project Scope

1.1 System Overview

We plan to implement DynamoDB, a distributed key-value storage system inspired by Amazon's Dynamo, designed to provide high availability, scalability, and low latency with a primary focus on fault tolerance and graceful failure handling.

1.2 Key Features

Key-Value Storage: The system will provide simple PUT and GET operations for storing and retrieving data using unique keys, with operations confined to single data items.

Data Partitioning and Replication: Data will be distributed across multiple nodes using consistent hashing and replicated N times across different nodes to ensure durability and availability even during failures.

Quorum-Based Consistency: The system will implement tunable consistency using parameters (N, R, W) where $R + W \leq N$ ensures reads observe previous writes while tolerating node failures.

Versioning with Vector Clocks: Vector clocks will track version history across replicas, detect concurrent updates during network partitions, and enable proper conflict resolution.

Decentralized Architecture: Every node will have symmetric responsibilities with no central coordinator, eliminating single points of failure and allowing any node to handle requests for any key.

Routing Table Management: Each node will maintain a routing table mapping key ranges to responsible nodes, structured as a dictionary where keys represent range starts and values contain node information for coordinator identification.

Dynamic Membership: Nodes will be able to join or leave dynamically without downtime, with automatic redistribution of responsibilities through consistent hashing.

Gossip-Based Communication: Nodes will exchange routing information periodically using gossip protocol, ensuring updates propagate in $O(\log n)$ steps without centralized coordination.

2 Fault Tolerance: Core Focus

Fault tolerance will be the primary enhancement and focus of our DynamoDB implementation, designed to handle various failure scenarios while maintaining sys-

tem availability and eventual consistency.

2.1 Replication Strategy

Asynchronous Replication: When a write request is received, data will be written to W nodes synchronously before responding to the client, while remaining $(N-W)$ replicas will be updated asynchronously to achieve eventual consistency within finite time.

Preference List: Each key will be assigned to N nodes forming its preference list, ensuring that data remains available even if multiple nodes fail simultaneously, tolerating up to $(W-1)$ failures for writes and $(N-R)$ failures for reads.

2.2 Handling Network Partitions

Write Availability During Partitions: The system will accept writes on both sides of a network partition, prioritizing availability over strong consistency and allowing operations to continue despite network splits.

Partition Healing and Reconciliation: After network partitions heal, a dedicated reconciliation thread will compare data across replicas and resolve conflicts using vector clocks and timestamps to restore consistency.

2.3 Node Failure Management

Temporary Failure Handling: The system will track potentially recoverable nodes separately from active nodes, periodically checking their status and allowing automatic reintegration without manual intervention.

Request Routing During Failures: When a coordinator node fails, requests will be automatically forwarded to the next available node in the preference list, ensuring uninterrupted service.

Permanent Failure Recovery: When nodes permanently fail, their data will remain available on remaining replicas, and responsibilities will be redistributed through consistent hashing without service disruption.

2.4 Continuous Reconciliation

Background Reconciliation Thread: A dedicated thread on each node will continuously compare data across replicas, proactively detecting and resolving inconsistencies to minimize divergence time.

Read Repair Mechanism: During GET operations, coordinators will identify stale replicas using vector clock comparisons and asynchronously update them after responding to clients, piggybacking consistency maintenance on read operations.

2.5 Failure Detection

Gossip-Based Detection: Nodes will use gossip protocol to exchange membership and liveness information, detecting failures through periodic heartbeats without requiring centralized monitoring.

Timeout-Based Identification: Nodes that fail to respond within configured timeouts will be marked as potentially failed, triggering routing table updates and request redirection to available nodes.

3 Challenges

Eventual Consistency: Achieving consistency across replicas during network partitions when conflicting writes occur and ensuring convergence after partition healing through reconciliation.

High Availability for Writes: Ensuring write operations succeed even during node failures or network partitions without violating the "always-on" requirement.

Handling Failures: Distinguishing between temporary and permanent failures, continuing operations during temporary unavailability, and recovering from permanent failures without service interruption.

Low Latency: Providing single-digit millisecond response times through efficient routing and optimized quorum operations even during partial failures.

Scalability: Scaling efficiently as nodes and data volume increase while maintaining fault tolerance guarantees and uniform load distribution.

Failure Detection: Reliably detecting node failures and network partitions without false positives in a decentralized manner.

Routing Table Consistency: Maintaining consistent topology views across all nodes without centralized coordination when membership changes occur.

4 Solutions and Techniques

4.1 Consistent Hashing for Partitioning

Consistent hashing will map keys and nodes onto a virtual ring, ensuring only K/N keys are redistributed when nodes join or leave, providing incremental scalability and automatic load redistribution during failures.

4.2 Vector Clocks for Versioning

Vector clocks will detect causality between versions by comparing version vectors—dominated versions will be discarded while causally unrelated versions will indicate concurrent updates requiring reconciliation.

4.3 Quorum-Based Replication

Configuring W and R such that $R + W \leq N$ will ensure read-write overlap for consistency while allowing the system to tolerate up to $(W-1)$ write failures and $(N-R)$ read failures.

4.4 Gossip Protocol for Decentralization

Nodes will periodically exchange routing and membership information with random peers, propagating updates in $O(\log n)$ rounds while detecting failures and maintaining symmetric responsibilities.

4.5 Reconciliation for Eventual Consistency

A background reconciliation thread will continuously compare replicas and resolve conflicts using timestamps for syntactic reconciliation, ensuring convergence after partitions or failures.

4.6 Read Repair Mechanism

During GET operations, coordinators will identify stale replicas using vector clock comparisons and asynchronously update them, improving consistency without impacting read latency.

4.7 Coordinator-Based Request Handling

Each request will be managed by a coordinator from the key's preference list, which will collect responses, detect conflicts, ensure quorum requirements, and handle coordinator failures through preference list fallback.

4.8 Dynamic Membership Handling

New nodes will calculate their ring position and fetch key ranges from successors, while failed nodes will have their responsibilities automatically redistributed to remaining replicas through consistent hashing.

5 Testing Strategy

We will employ an interactive CLI-driven testing framework with iptables-based network fault injection to validate fault tolerance under real-world challenges. The framework will dynamically manage nodes while simulating network partitions to test system resilience. **Fault Tolerance Tests** will use iptables to create network partitions, validating that writes continue during splits and replicas reconcile after healing. **Scalability Tests** will verify consistent hashing during node changes. **Consistency Tests** will execute PUT/GET under failures to verify quorum-based replication and vector clocks. **Availability Tests** will ensure "always-on" guarantees with node failures. Multi-workload testing will validate different conflict resolution paths.

6 Conclusion

Our DynamoDB implementation will demonstrate a highly available, fault-tolerant distributed key-value store using consistent hashing, vector clocks, quorum-based replication, and gossip protocols to handle network partitions, node failures, and concurrent updates while maintaining operational resilience.